

แบบทดสอบท้ายแล็บ

Custom ViewResolver ใน Spring Boot + Thymeleaf

วิชา CP353002 Principles of Software Design

ชื่อ-นามสกุล: นางสาวสโรชา เสาทอง รหัสนักศึกษา: 673380296-7

คำชี้แจง: ส่วนที่ 1 ปรนัย เลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว ส่วนที่ 2 อธิบาย ตอบเป็นข้อความอธิบาย ส่วนที่ 3 ปฏิบัติ
ให้ลงมือแก้โค้ดจริงในโปรเจกต์ของตัวเอง แล้ว capture หน้าจอผลลัพธ์แปะในกล่องที่เตรียมไว้

ส่วนที่ 1 — ปรนัย (Multiple Choice)

1. ในสถาปัตยกรรม MVC ส่วนใดที่ทำหน้าที่แปลง "ชื่อ view" ให้กลายเป็น path ของไฟล์ view จริง? (2 คะแนน)

- ก. Model
- ข. Controller
- ค. ViewResolver
- ง. DispatcherServlet

2. เมื่อ Controller เขียน return "home"; คำ "home" ที่ถูกส่งกลับคืออะไร? (2 คะแนน)

- ก. Path ของไฟล์ HTML บนเครื่อง server
- ข. ชื่อ view ให้ Controller (logical view name)
- ค. ชื่อ bean ของ ViewResolver
- ง. ชื่อ method ของ Controller

3. component ใดใน Spring MVC เป็นผู้เรียกใช้ ViewResolver โดยอัตโนมัติ หลังจาก Controller คืนค่าชื่อ view? (2 คะแนน)

- ก. HandlerMapping
- ข. DispatcherServlet
- ค. SpringTemplateEngine
- ง. HomeController

4. ใน ThymeleafConfig.java เมธอด resolver.setPrefix("classpath:/custom-templates/") ทำหน้าที่อะไร? (2 คะแนน)

- ก. กำหนด URL ที่ Controller จะรับ request
- ข. กำหนดโฟลเดอร์ต้นทางที่ ViewResolver จะค้นหาไฟล์ template
- ค. กำหนดชื่อ database connection
- ง. กำหนด port ที่แอปพลิเคชันรัน

5. ถ้ามีการลงทะเบียน ViewResolver มากกว่าหนึ่งตัวในบริบท (context) เดียวกัน Spring จะเลือกใช้ตัวไหนก่อน? (2 คะแนน)

- ก. ตัวที่ประกาศเป็นตัวแรกในไฟล์เสมอ
- ข. ตัวที่มีค่า order ต่ำกว่า (ถูกเรียกก่อน)

ค. สุ่มเลือก

ง. ตัวที่ตั้งชื่อ bean เรียงตามตัวอักษร

6. เพราะเหตุใด Controller ในตัวอย่างนี้จึงใช้ @Controller แทน @RestController? (2 คะแนน)

ก. เพราะ @RestController ใช้กับ Thymeleaf ไม่ได้เลย

ข. เพราะต้องการให้ค่าที่ return ถูกตีความเป็นชื่อ view ไม่ใช่ response body โดยตรง

ค. เพราะ @Controller ทำงานเร็วกว่า

ง. ไม่มีความแตกต่างกันเลย

1. ใครเป็นผู้เรียกใช้ Custom ViewResolver และเรียกเมื่อไร? จงอธิบายเป็นลำดับขั้นตอน ตั้งแต่ Browser ส่ง request จนได้ HTML response กลับ (5 คะแนน)

ผู้เรียกใช้: DispatcherServlet (ตัว front controller ของ Spring MVC) เป็นผู้เรียกใช้ ViewResolver ไม่ใช่ HomeController เรียกเอง

ลำดับขั้นตอนตั้งแต่ Browser ส่ง request จนได้ HTML response กลับ:

1. Browser ส่ง HTTP request เช่น GET `http://localhost:8080/` ไปยัง embedded Tomcat
 2. DispatcherServlet รับ request เนื่องจากเป็น servlet เดียวที่ถูก map ให้รับทุก request (ตั้งค่าอัตโนมัติผ่าน `@SpringBootApplication`)
 3. DispatcherServlet หา Handler ที่เหมาะสม โดยใช้ HandlerMapping เพื่อจับคู่ URL / กับ method `home()` ใน HomeController (ที่มี `@GetMapping("/")`)
 4. HandlerAdapter เรียก method `home()` ใน HomeController — method นี้ทำการ `model.addAttribute("message", ...)` แล้ว return String `"home"` ซึ่งเป็นเพียง *logical view name* ไม่ใช่ path ไฟล์จริง
 5. DispatcherServlet รับค่า `"home"` กลับมา และเนื่องจากไม่ใช่ `@ResponseBody/@RestController` จึงตีความว่าค่านี้คือชื่อ view ที่ต้องแปลงเป็นหน้าจริง
 6. DispatcherServlet เรียก `ViewResolver.resolveViewName("home", locale)` — ตรงนี้คือจุดที่ custom `ThymeleafViewResolver` ที่เรากำหนดใน `ThymeleafConfig` ถูกเรียกใช้งาน (เพราะถูกลงทะเบียนเป็น Bean และตั้ง `order(1)`)
 7. ViewResolver ใช้ `SpringResourceTemplateResolver` ที่ผูกไว้ ประกอบ path จาก prefix (`"classpath:/custom-templates/"`) + ชื่อ view (`"home"`) + suffix (`".html"`) → ได้ `classpath:/custom-templates/home.html` แล้วคืนค่าเป็น object View (ของ Thymeleaf) กลับไปให้ DispatcherServlet
 8. DispatcherServlet เรียก `view.render(model, request, response)` — ตรงนี้ Thymeleaf engine จะอ่านไฟล์ `home.html` จริง แล้วแทนที่ expression เช่น `th:text="${message}"` ด้วยค่าจาก Model ที่ Controller ใส่ไว้
 9. ผลลัพธ์ HTML ที่ render เสร็จแล้ว ถูกเขียนลงใน `HttpServletResponse` แล้วส่งกลับไปยัง Browser
 10. Browser รับ HTML response และแสดงผลข้อความ `"Hello from Thymeleaf with a custom ViewResolver!"`
-

2. จงอธิบายว่าเหตุใดการแยก ViewResolver ออกจาก Controller จึงสอดคล้องกับหลักการ separation of concerns และ dependency inversion ใน Principles of Software Design (5 คะแนน)

Separation of Concerns:

- Controller มีหน้าที่รับผิดชอบเฉพาะ *business/request-handling logic* — รับ request, ประมวลผล, เตรียมข้อมูลใส่ Model, ตัดสินใจว่าจะแสดงผลด้วย view ชื่ออะไร
- ViewResolver มีหน้าที่รับผิดชอบเฉพาะ *การแปลงชื่อ view เป็น resource* จริง (path, suffix, template engine ที่ใช้)
- เมื่อแยกหน้าที่ทั้งสองออกจากกันชัดเจน แต่ละ component จึงมีเหตุผลเดียวในการเปลี่ยนแปลง (Single Responsibility) — ถ้าจะเปลี่ยนตำแหน่งไฟล์ template หรือเปลี่ยน engine ก็แค่ ThymeleafConfig โดยไม่กระทบ HomeController เลย

Dependency Inversion:

- Controller (module ระดับสูง) ไม่ได้ขึ้นกับรายละเอียดการ implement ของการ resolve view (module ระดับล่าง) โดยตรง แต่ขึ้นกับ **abstraction** คือ "ชื่อ view เจริญระกะ" (String เช่น "home") เท่านั้น
- ทั้ง Controller และ ViewResolver ต่างก็พึ่งพา "สัญญา" (contract) ร่วมกันผ่านกลไกของ Spring MVC (interface ViewResolver) ไม่ใช่ Controller ไปเรียกใช้ SpringResourceTemplateResolver โดยตรง
- ผลคือสามารถสลับ **implementation** ของการ resolve view ได้อย่างอิสระ (เปลี่ยน path, เปลี่ยนจาก Thymeleaf เป็น FreeMarker/JSP, หรือเพิ่ม ViewResolver หลายตัวพร้อม order) โดยไม่ต้องแก้โค้ดใน Controller แม้แต่บรรทัดเดียว — เป็นไปตามหลักที่ว่า "high-level module ไม่ควรขึ้นกับ low-level module โดยตรง ทั้งคู่ควรขึ้นกับ abstraction"

3. หากต้องการเพิ่ม view ใหม่ชื่อ "about" โดยยังใช้ custom ViewResolver ตัวเดิม ต้องสร้างหรือแก้ไขไฟล์ใดบ้าง? ระบุชื่อไฟล์และ path ให้ครบถ้วน (3 คะแนน)

ต้องแก้ไข/สร้างทั้งหมด 2 ไฟล์ (ไม่ต้องแก้ ThymeleafConfig.java เพราะ ViewResolver ตัวเดิมใช้ prefix/suffix เดิมได้กับทุก view):

1. แก้ไขไฟล์: `src/main/java/com/example/demo/controller/HomeController.java`

- เพิ่ม method ใหม่ในคลาสเดิม:

```
@GetMapping("/about")
```

```
public String about(Model model) {
```

```
    model.addAttribute("message", "This is the About page");
```

```
    return "about";
```

```
}
```

2. สร้างไฟล์ใหม่: src/main/resources/custom-templates/about.html

- o เนื้อหาโครงเดียวกับ home.html เช่น:

html

```
<!DOCTYPE html>
```

```
<html xmlns:th="http://www.thymeleaf.org">
```

```
<head><meta charset="UTF-8"><title>About</title></head>
```

```
<body>
```

```
<h1 th:text="${message}">Default message</h1>
```

```
</body>
```

```
</html>
```

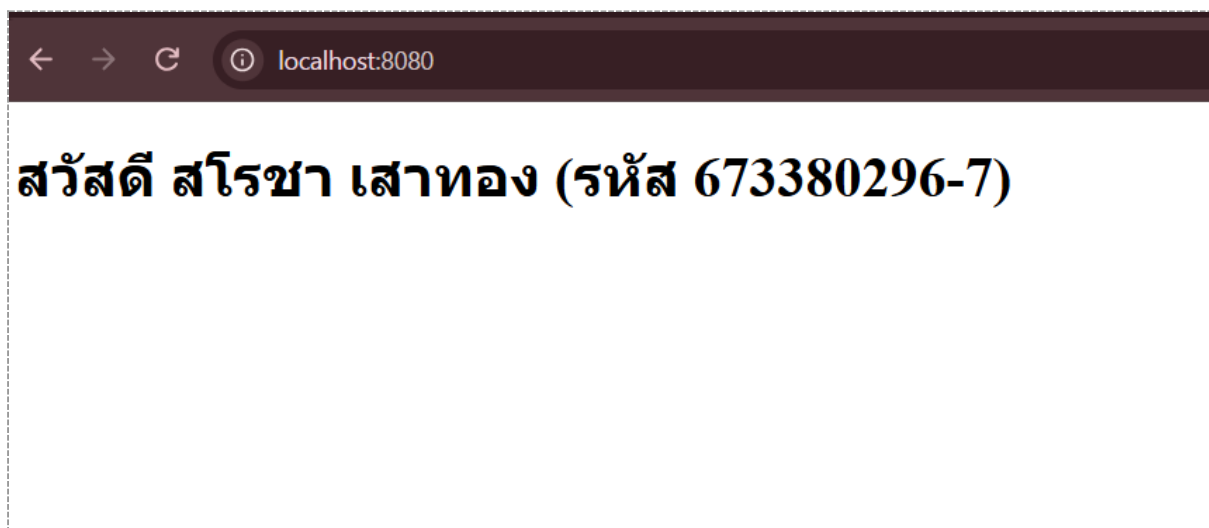
ส่วนที่ 3 — ปฏิบัติ (แก้โค้ดจริง + แบนภาพหน้าจอ)

ทำตามคำสั่งแต่ละข้อในโปรเจกต์ของตัวเอง รันด้วย `mvn spring-boot:run` แล้ว capture หน้าจอผลลัพธ์และในกล่องเส้นประด้านล่างแต่ละข้อ

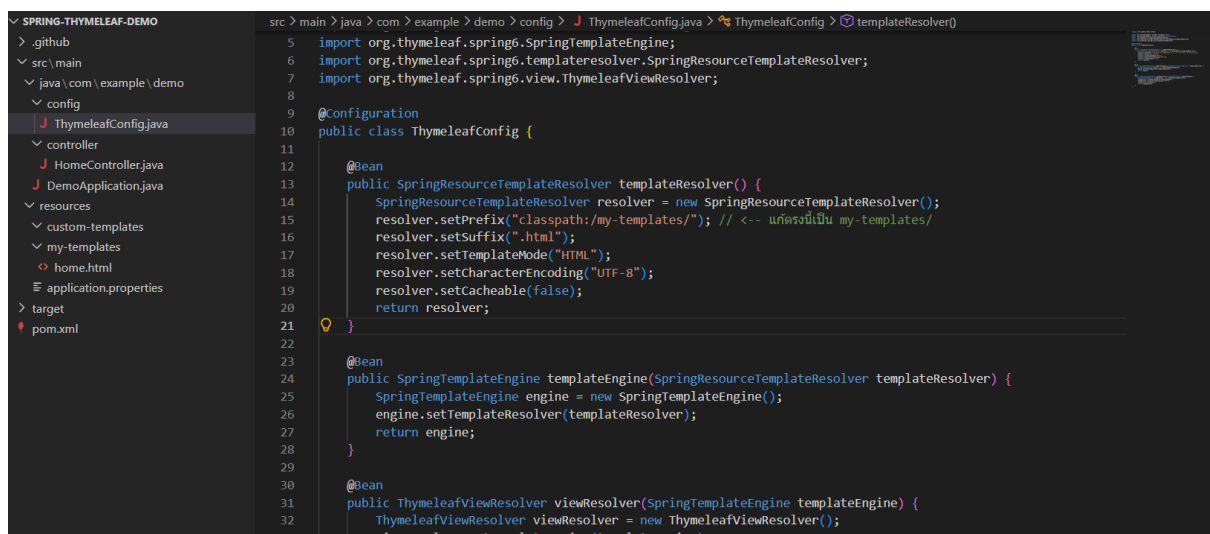
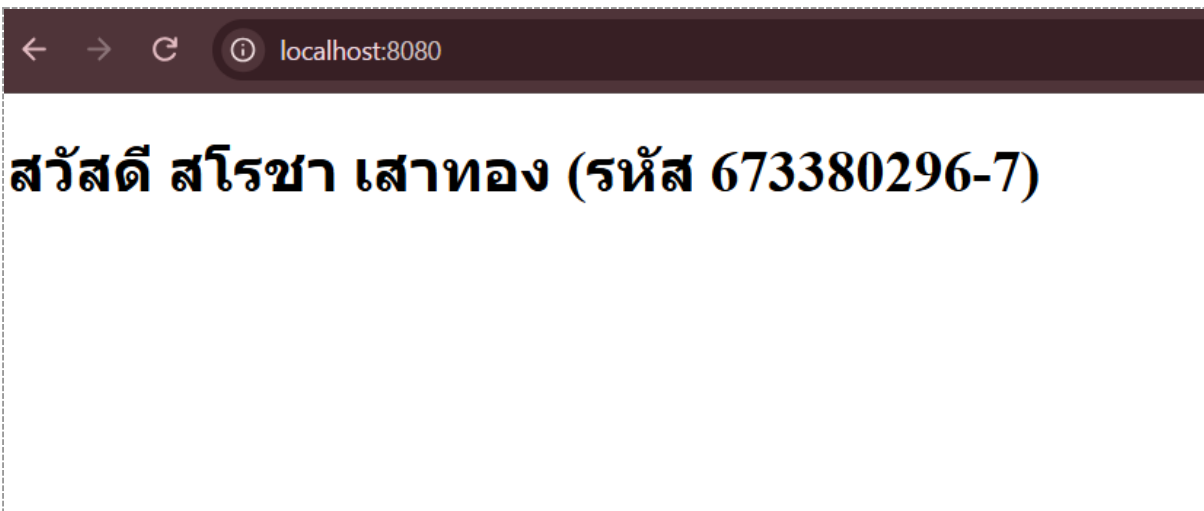
1. แก้ไขค่าตัวแปร `message` ใน `HomeController.java` ให้แสดงชื่อ-นามสกุลของตัวเองแทนข้อความเดิม แล้ว capture หน้าเว็บ `http://localhost:8080/` ที่แสดงชื่อของตัวเอง (3 คะแนน)



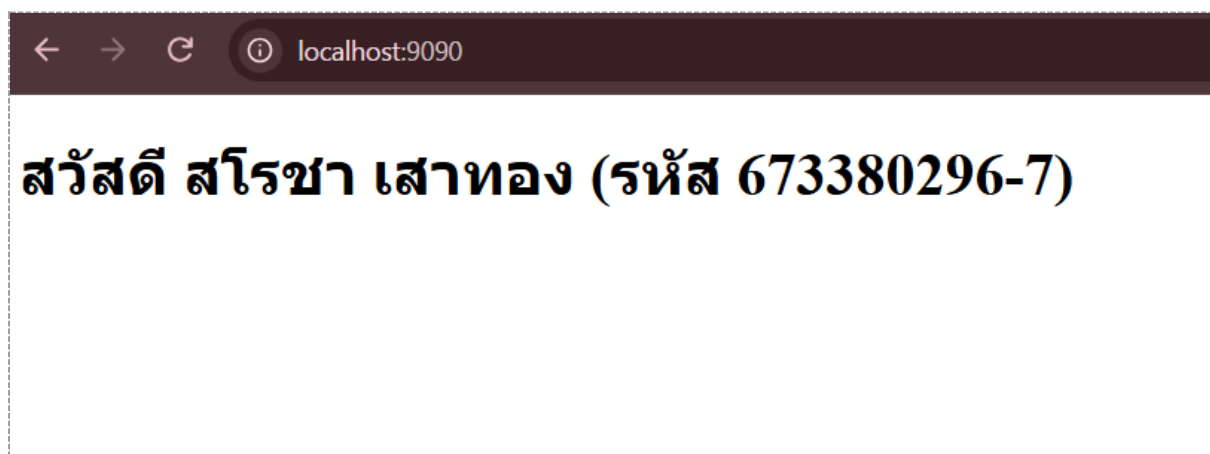
2. เพิ่มตัวแปรใหม่ชื่อ `studentId` ส่งจาก Controller ไปแสดงต่อจากชื่อในหน้า `home.html` (เช่น "สวัสดี ชื่อ (รหัส XXX)") แล้ว capture หน้าเว็บที่แสดงผลลัพธ์ (3 คะแนน)



3. เปลี่ยนค่า `resolver.setPrefix()` ใน `ThymeleafConfig.java` ให้ชี้ไปโฟลเดอร์ใหม่ชื่อ `my-templates/` แทน `custom-templates/` (ต้องย้ายไฟล์ `home.html` ไปด้วย) แล้ว capture ทั้งโค้ดที่แก้ และหน้าเว็บที่ยังทำงานปกติ (4 คะแนน)



4. เปลี่ยน server.port ใน application.properties เป็น 9090 แล้ว capture หน้าเว็บที่ URL เปลี่ยนเป็น <http://localhost:9090/> (3 คะแนน)



5. เพิ่ม view ใหม่ชื่อ "about" (เพิ่ม @GetMapping("/about") และไฟล์ about.html)
ให้แสดงข้อความแนะนำตัวสั้นๆ แล้ว capture หน้าเว็บ /about ที่แสดงผลลัพธ์ (4 คะแนน)

